



HIT391 – MACHINE LEARNING: ADVANCEMENTS AND APPLICATIONS

GROUP ASSIGNMENT 3



Table of Contents

1. Introduction.....	3
2. Dataset Description.....	3
3. Task 1 - Clustering Algorithms	5
3.1 Objective.....	5
3.2 Model Selection.....	5
3.3 Implementation steps	5
a. Data Preprocessing	5
b. Data Splitting	10
c. Model Implementation	10
d. Model Training.....	11
e. Model Tuning.....	17
f. Model Evaluation.....	19
4. Task 2 - Classifier Algorithms on Titanic Dataset	21
4.1 Objective.....	21
4.2 Model Selection.....	21
4.3 Implementation steps	21
4.3.1. Data Preprocessing	22
We created bar plots to show the number of missing values in each column for both the training and test sets. This made it clear that there was a significant amount of missing data in the 'Cabin' column for both datasets, with the training set lacking 687 values and the test set missing 327. The 'Age' feature also had a considerable number of missing entries, while the 'Embarked' feature in the training set and 'Fare' in the test set had a few gaps as well.	
b. Handling Missing Values	23
c. Feature Engineering:.....	26

4.3.2 Data Splitting	27
4.3.3. Model Implementation	29
Logistic Regression	30
4.3.4. Model Training	36
4.3.5 Model Tuning	40
4.3.6 Model Evaluation	46
5. Conclusion	50
References:	51
Figure 1 - Plot the Elbow Graph to find the best k	12
Figure 2 - Elbow Graph	13
Figure 3 - Model training using K-Means.....	14
Figure 4 - K-Means Clustering in 2D	15
Figure 5 - Dendrogram on Train Dataset.....	17
Figure 6 - Model Evaluation	20

1. Introduction

This project aimed to apply machine learning techniques to two distinct tasks involving real-world datasets. The first task focused on clustering individuals based on their hobbies and interests using unsupervised learning methods. The second task required the implementation of classification algorithms to predict survival outcomes in the Titanic shipwreck scenario, a classic supervised learning problem. These tasks provided hands-on experience with end-to-end data science workflows, including data preprocessing, model development, hyperparameter tuning, and evaluation. The primary goal was to demonstrate the ability to select appropriate models, apply them correctly, and interpret the results meaningfully.

2. Dataset Description

Task 1 Dataset: kaggle_Interests_group.csv

This dataset contains binary interest indicators for 6340 individuals across 217 hobby and interest categories. Each column (feature) represents a specific interest (e.g., sports, music, travel), encoded as 1 for interested or 0 for not interested. The dataset has no labels or identifiers, making it ideal for unsupervised clustering tasks. The objective was to identify natural groupings of people based on shared interests using K-Means and Hierarchical Clustering.

Task 2 Datasets: Titanic - Machine Learning from Disaster

This task used three datasets from the Kaggle Titanic competition:

- **train.csv:** Contains 891 passenger records with labels indicating whether each passenger survived (Survived = 1) or not (Survived = 0). Features include demographic and ticketing details such as Age, Sex, Pclass, Fare, and Embarked.

- test.csv: Contains 418 unlabeled passenger records used for final predictions.
- gender_submission.csv: A sample submission file provided by Kaggle, representing a naive model that predicts all female passengers survived and all males did not.

3. Task 1 - Clustering Algorithms

3.1 Objective

The main goal of Task 1 was to explore unsupervised learning techniques to cluster individuals based on their hobbies and interests. The dataset included 6340 individuals and 217 binary features representing whether each person had a specific interest.

3.2 Model Selection

Two clustering algorithms were implemented:

- K-Means Clustering
- Hierarchical Clustering

3.3 Implementation steps

a. Data Preprocessing

The first stage of any machine learning task is to prepare the data to make sure that it is clean, consistent, and suitable for model training. In this project, we used the dataset `kaggle_Interests_group.csv`, which contains records related to user interests on a social media or community platform. The preprocessing process followed some main steps: importing the dataset, handling categorical variables, standardizing numerical features, and ensuring data cleanliness.

In the first step of our project, we performed data preprocessing to prepare the dataset for clustering. The dataset used in this assignment was `kaggle_Interests_group.csv`, which contains categorical attributes representing different user interests and behavior patterns.

We used the pandas library to load the dataset into a DataFrame and examined its dimensions and structure:

1) Data Preprocessing

```
# Load the dataset
df = pd.read_csv("kaggle_Interests_group.csv")
print("Original dataset shape:", df.shape)
```

Python

```
... Original dataset shape: (6340, 219)
```

We then checked the shape of the dataset using `df.shape` and looked at a few rows using `df.head()`. This helped us understand what the dataset looked like and what kind of data we were working with. Most of the columns were text-based (categorical), which meant we needed to convert them to numbers before we could train a model.

To get an overview of the dataset, we used several functions such as `.describe()`, `.head()`, `.tail()`, and `.info()` to inspect feature types, missing values, and potential outliers. We also checked for duplicate rows using `.duplicated().sum()` and removed them with `.drop_duplicates()` to avoid biasing the clustering.

```
# Drop Duplicates if any
df = df.drop_duplicates()
print("Shape after removing duplicates:", df.shape)
```

Python

Shape after removing duplicates: (6181, 219)

Next, we examined missing values in the dataset. We calculated the total number of missing values per column, and then computed the overall sum to understand the scale of the issue.

```
# Total missing data
missing_total = df.isnull().sum().sum()
print("Total missing values:", missing_total)
```

Python

We then removed columns that had more than 1000 missing values using a custom-defined function. This step was important to eliminate features that might distort the clustering due to sparsity.

```
# Drop columns who have more than 5000 missed values

def drop_missing_values_columns(df , threshold) :
    list_of_index = []
    columns= list(df.isna().sum().index)
    for i in range (len(list(df.isna().sum().values))) :
        if list(df.isna().sum().values)[i]>threshold :
            list_of_index.append(i)
    list_of_columns_to_drop = []
    for i in (list_of_index) :
        list_of_columns_to_drop.append(columns[i])
    df.drop(list_of_columns_to_drop , axis = 1 , inplace = True)
    return df
```

Python

After removing those columns, we filled the remaining missing values with the **median** of each column. We excluded the "group" column from this process since it was later encoded separately.

```
# fillna the missing values with the median
columns = list(df1.columns)
columns.remove("group")
for col in columns :
    median = df1[col].median()
    df1[col].fillna(1-median , inplace = True)
```

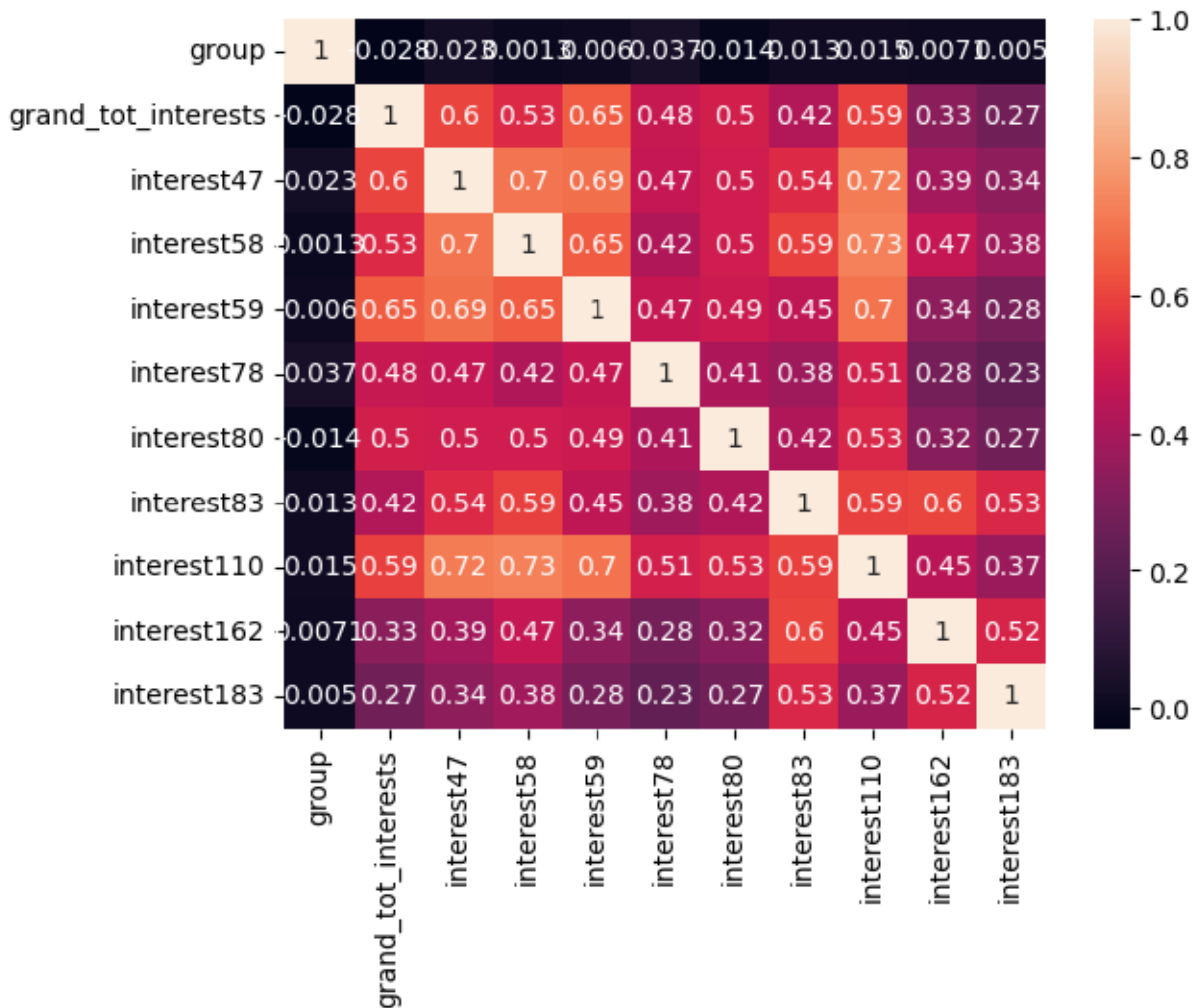
Python

The "group" column contained categorical values, so we applied LabelEncoder to convert it into numeric form.

```
# Encoding categorical data, the first column "group" is categorical
le=LabelEncoder()
df1["group"]=le.fit_transform(df1["group"] )
```

Python

To understand how features related to each other, we calculated and visualised a **correlation matrix**. This helped us see which interest categories had strong or weak relationships.



From the heatmap, we can see that many features had high pairwise correlations, among interest-related fields. This shows that users who were interested in one topic often had related interests in others too. The "group" column showed very low correlation with the other features, confirming its use as a meaningful category to separate clusters.

After the heatmap, we selected three features: "group", "grand_tot_interests", and "interest47" for further analysis. These were chosen based on their statistical importance and visual relevance.

```
df2 = df1[["group" , "grand_tot_interests" , "interest47"]]  
df2
```

Python

	group	grand_tot_interests	interest47
0	0	17	1.0
1	0	43	1.0
2	0	27	1.0
3	0	34	1.0
4	0	36	1.0
...	...	...	...
6335	1	53	1.0
6336	1	48	1.0
6337	1	48	1.0
6338	1	38	1.0
6339	1	66	1.0

6181 rows × 3 columns

The "grand_tot_interests" column was then normalised between 0 and 1 using MinMaxScaler:

```
# Using MinMaxScaler from sklearn.preprocessing to normalise the grand_tot_interests column in df2.  
from sklearn.preprocessing import MinMaxScaler  
mms = MinMaxScaler()  
df2["grand_tot_interests"] = mms.fit_transform(df2[["grand_tot_interests"]])
```

Python

Finally, we have all features in the cleaned dataset using StandardScaler. This ensures that clustering algorithms don't get affected by feature scale differences.

```
# Standardize the data  
scaler = StandardScaler()  
df_scaled = scaler.fit_transform(df1)
```

Python

b. Data Splitting

Although clustering is an unsupervised learning task, we still split the dataset into three subsets: training, evaluation, and test sets. This mimics real-world workflows and allows us to test model robustness.

```
X_train, X_temp = train_test_split(df_scaled, test_size=0.4, random_state=42)
X_eval, X_test = train_test_split(X_temp, test_size=0.5, random_state=42)
print("Train shape:", X_train.shape)
print("Eval shape:", X_eval.shape)
print("Test shape:", X_test.shape)
```

Python

We will split into Train (60%), Eval (20%), Test (20%)

X_train: 60% of the dataset, used to train clustering models

X_eval: 20% of the dataset, used to evaluate model quality (e.g., silhouette score)

X_test: 20% reserved for final testing or extra visualisation

This split ensured that we could tune and evaluate models without overfitting on the full dataset.

```
Train shape: (3708, 11)
Eval shape: (1236, 11)
Test shape: (1237, 11)
```

c. Model Implementation

In this part of the assignment, we selected the clustering algorithms that would be used to group users based on their interests. We focused on implementing two models: **KMeans Clustering** and **Agglomerative (Hierarchical) Clustering**. This step involved importing the necessary libraries and preparing the model structures, but no model training or prediction was done here — those are covered in Part d.

We began by importing all essential libraries required for clustering, scaling, plotting, encoding, and evaluation. The code below was used to set up our environment for implementation:

```
# Import all the necessary libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.preprocessing import LabelEncoder
from sklearn.cluster import KMeans, AgglomerativeClustering
from sklearn.metrics import silhouette_score
import seaborn as sns
import scipy.cluster.hierarchy as sch
```

Python

These imports were performed to support the entire workflow, including:

- **Pandas & NumPy**: for data handling and manipulation
- **Matplotlib & Seaborn**: for visualisations (used later)
- **Scikit-learn modules**: for scaling (StandardScaler), encoding (LabelEncoder), and clustering (KMeans, AgglomerativeClustering)
- **Scipy**: to create hierarchical dendrograms

Later in the notebook, KMeans was re-imported in a dedicated cell, which we preserved exactly:

```
# We will use sklearn to implement the K-Means model

from sklearn.cluster import KMeans
```

Python

d. Model Training

We will train the model using the training dataset.

* Clustering Method 1: K-MEANS Clustering

Once the data was preprocessed, normalised, and split, we trained the K-Means clustering model using the scaled training dataset derived from the kaggle_Interests_group.csv file. Below is a breakdown of the steps involved:

- Training dataset size: 60% of the total dataset.
- Algorithm used: KMeans from sklearn.cluster.
- Hyperparameter – n_clusters:

- The number of clusters was selected using the Elbow Method, which involved plotting the Within-Cluster Sum of Squares (WCSS) against various values of k .
- Based on the “elbow point” where the rate of decrease in WCSS slowed significantly, we selected $k = 5$ as the optimal number of clusters.

```
✓ [25] # We will use sklearn to implement the K-Means model
0s
from sklearn.cluster import KMeans

✓ # Plot Elbow Graph to find the best K based on the "elbow"
0s
import matplotlib.pyplot as plt
inertias = []

for i in range(2,12):
    kmeans = KMeans(n_clusters=i).fit(df2)
    inertias.append(kmeans.inertia_)
print("inertia = ", inertias)
plt.plot(range(2, 12), inertias, marker='o', color='green')
plt.title('Elbow method - Optimal k for KMeans')
plt.xlabel('Number of clusters (k)')
plt.ylabel('Inertia (Sum of Squared Distances)')
plt.show()

inertia = [2163.9449309888273, 1299.8758583872452, 653.909136642578, 408.82429942381594, 254.94352423394503, 146.50180079309527,
```

Figure 1 - Plot the Elbow Graph to find the best k

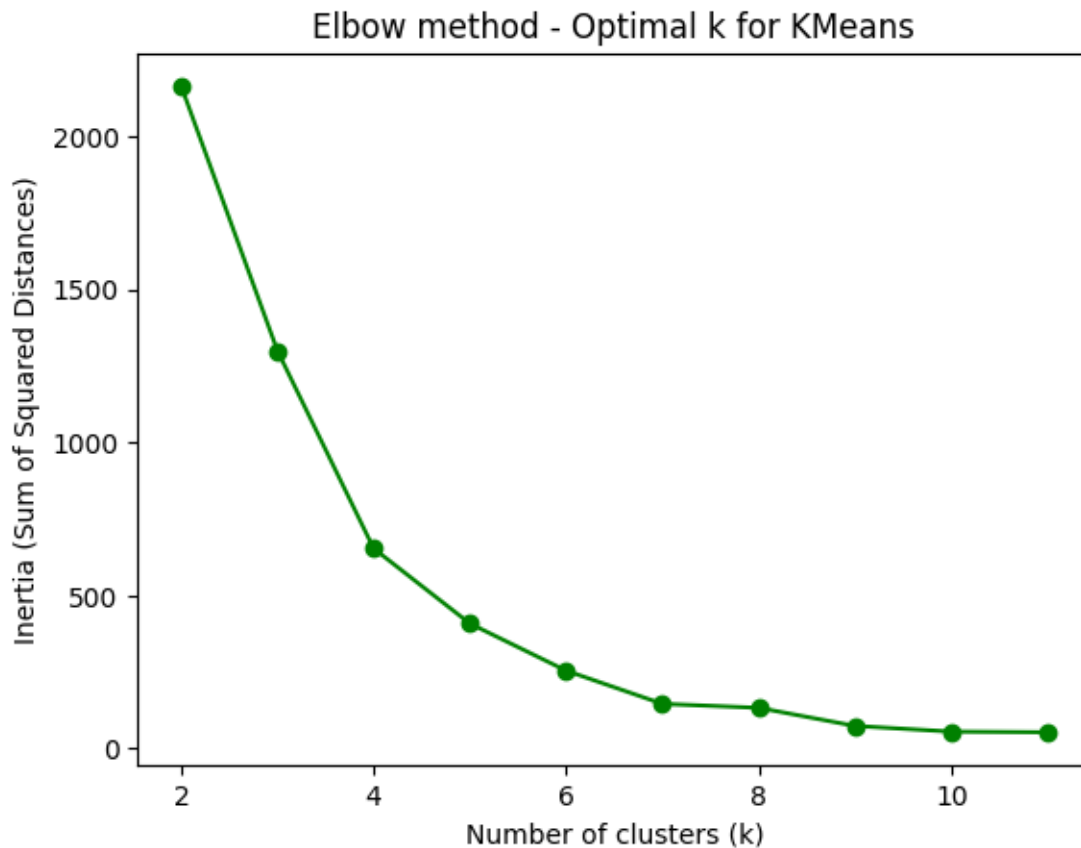


Figure 2 - Elbow Graph

Using $k = 5$, we trained the final model:

- The `KMeans(n_clusters=5, random_state=42)` model was fit using `.fit()` on the scaled training data.
- Cluster labels were assigned using `.predict()` and stored for further evaluation.

```
✓ [28] # Find the best k (number of clusters) for K-Means
0s    from kneed import KneeLocator
      kneedle = KneeLocator(range(2,12), inertias , curve = "convex" , direction = "decreasing")
      kneedle.elbow

→ np.int64(5)
```

So it is very clear that 5 is the best k , so we have 5 clusters.

```
✓ [50] # Best k=5 from elbow plot
0s    best_k = kneedle.elbow
      kmeans = KMeans(n_clusters=best_k, random_state=42)
      # Train the model
      kmeans.fit(X_train)
      # Predict labels for the training set
      cluster_labels = kmeans.labels_
```

Figure 3 - Model training using K-Means

Model Insights:

- The use of KneeLocator removed subjective interpretation in elbow analysis.
- The resulting 5 clusters provided a more refined grouping of user interests.
- This clustering structure was later evaluated using the Silhouette Score and PCA visualisations.

After training the K-Means model with $k = 5$, we visualised the clustering outcome in two dimensions using selected scaled features: interest47 (x-axis) and grand_tot_interests (y-axis). This visualisation helps provide an intuitive understanding of how the model grouped the data.

Key visual elements:

- Colored dots represent individual data points, colored by cluster label.
- Red "X" markers indicate the centroids of the 5 clusters.
- The x and y features were selected to allow visual separation, although the full dataset contains 217 dimensions.

Insights:

- The clusters are well-separated vertically, suggesting that `grand_tot_interests` played a stronger role in differentiating groups.
- Horizontally, we observe some overlap, indicating that clustering based solely on two features doesn't capture the complete distinction.
- The spread and concentration of points around centroids indicate that K-Means has successfully captured central tendencies for distinct user interest profiles.

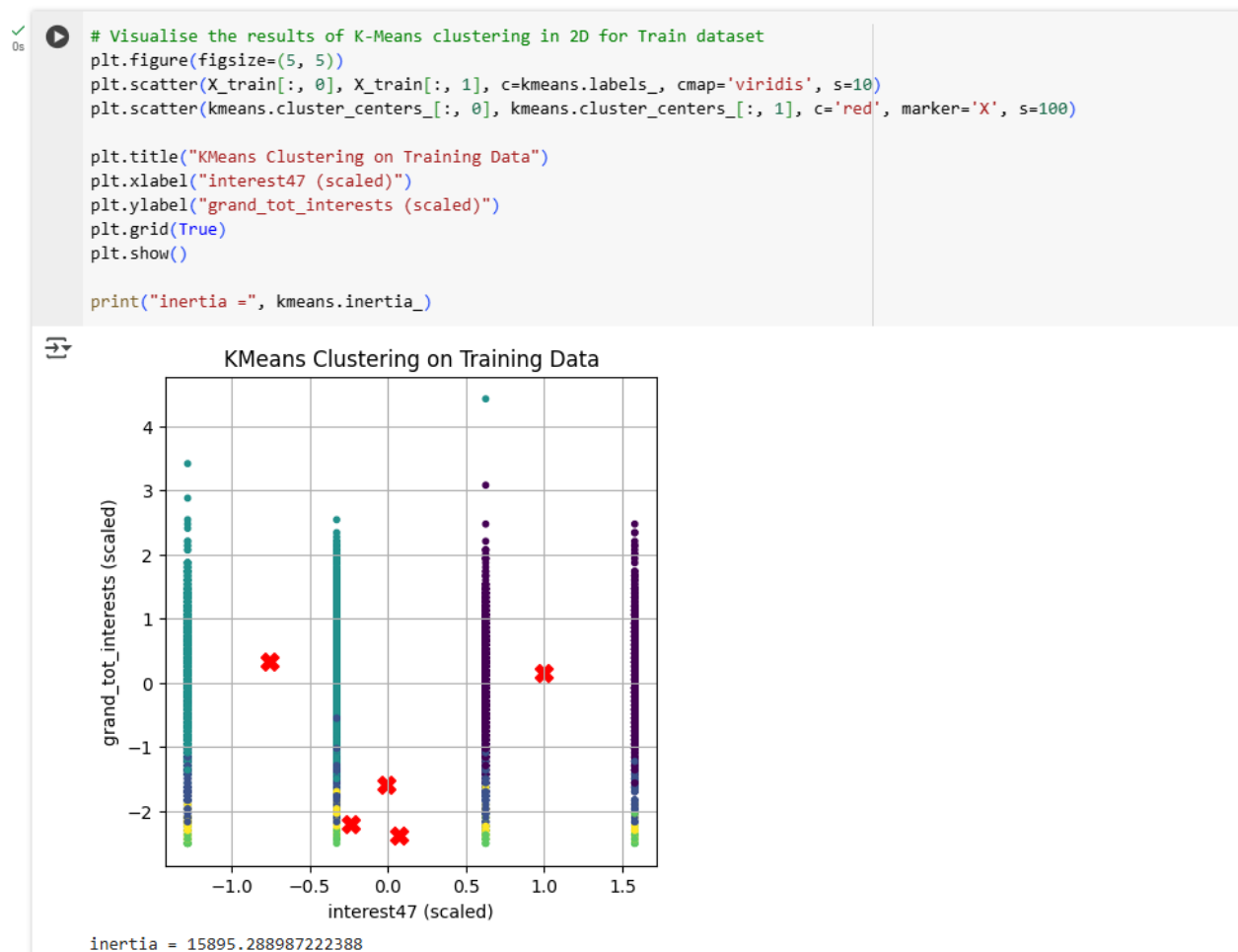


Figure 4 - K-Means Clustering in 2D

Model Inertia:

- The final inertia (sum of squared distances from each point to its cluster center) was: Inertia = 15895.29
- A lower inertia suggests tighter, more defined clusters.
- However, since inertia tends to decrease with increasing k, it is best interpreted in context with methods like the Elbow Curve and Silhouette Score.

* Clustering Method 2: Hierarchical (Agglomerative)

For the second clustering method, we used Agglomerative Hierarchical Clustering with the Ward linkage method on the scaled training dataset.

Dendrogram Analysis: To determine the optimal number of clusters, we first plotted a dendrogram using `scipy.cluster.hierarchy.dendrogram()`:

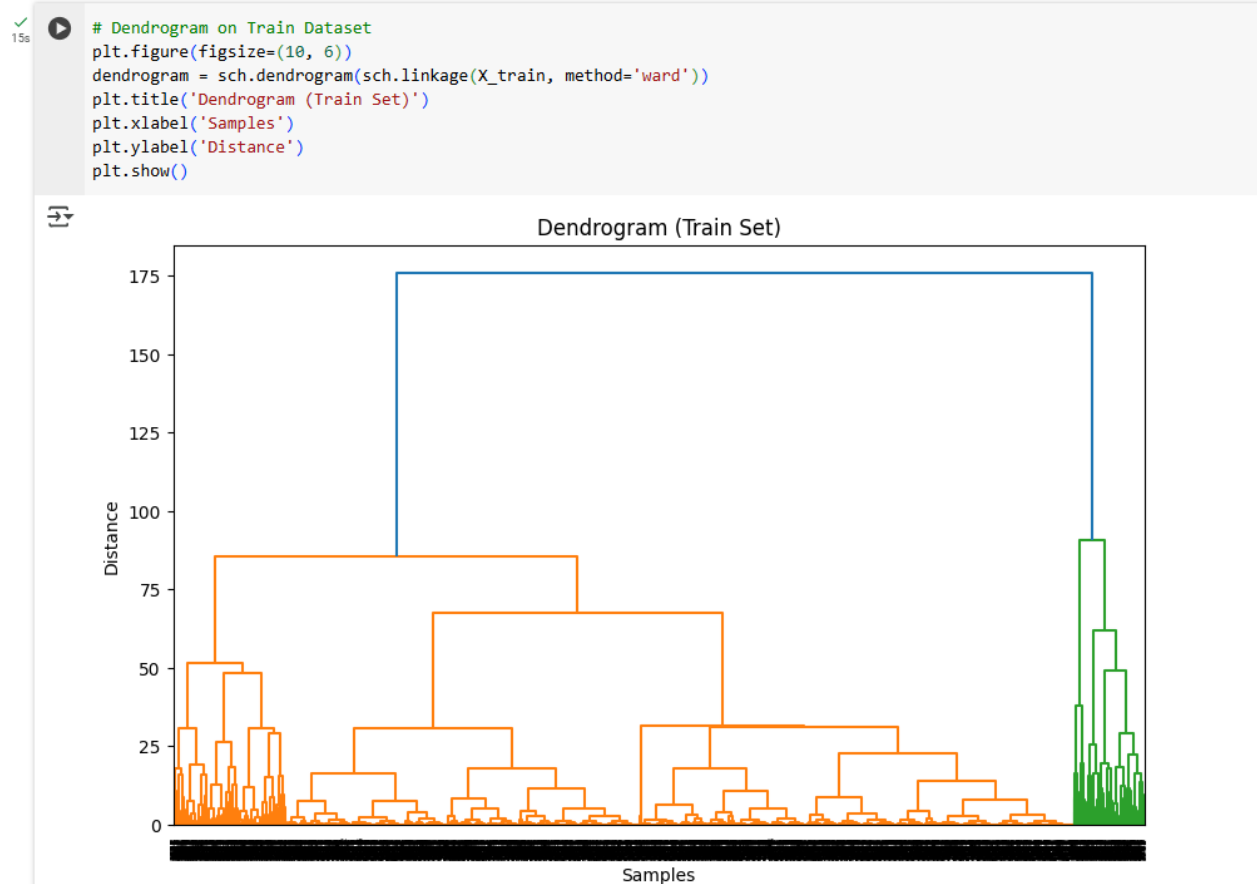


Figure 5 - Dendrogram on Train Dataset

- The dendrogram visually represents the hierarchy of cluster merges.
- By analyzing the vertical distance (height) between merged clusters, we can identify the ideal cutting point.
- From our dendrogram, the largest vertical gap without any horizontal cuts suggested that 5 clusters is a reasonable choice, consistent with our K-Means result.

Training the Model: We then fit the Agglomerative Clustering model using the selected cluster count

```
✓ [36] # Hierarchical Clustering
1s  hc_model = AgglomerativeClustering(n_clusters=5, linkage='ward')
    hier_labels = hc_model.fit_predict(X_train)
```

Insights:

- Unlike K-Means, this method does not rely on centroid calculation but instead on pairwise distances and merge strategies.
- The Ward linkage minimizes the variance within clusters during the merge process.
- The generated labels were used for evaluation (Silhouette Score) and visualized in 2D scatter plots.

e. Model Tuning

To improve clustering performance, we applied Grid Search over different hyperparameters for both K-Means and Hierarchical Clustering. The Silhouette Score was used as the primary metric to evaluate the separation quality of clusters.

* K-Means Clustering – Tuning n_clusters and init_method

We experimented with:

- Cluster values: 2 to 10
- Initialization methods: 'k-means++' and 'random'

Each combination was evaluated on the evaluation dataset, and the corresponding Silhouette Score was recorded.

Top 3 K-Means results:

n_clusters	init_method	Silhouette Score
2	k-means++	0.697
2	random	0.697
3	k-means++	0.641

Insight:

- The best result was achieved with 2 clusters using 'k-means++' initialization.
- However, we previously selected 5 clusters using the Elbow Method to balance interpretability and performance, even if the score was slightly lower (~0.42).

* Hierarchical Clustering – Tuning n_clusters and linkage

We tested:

- Linkage types: 'ward', 'complete', and 'average'
- Cluster range: 2 to 10

Top 3 Hierarchical Clustering results:

n_clusters	linkage	Silhouette Score
2	complete	0.799
2	average	0.799
3	complete	0.756

Insight:

- The complete linkage with 2 clusters gave the highest silhouette score (≈ 0.799).
- Similar to K-Means, the top-performing cluster count was low (2–3), but we retained 5 clusters in our final model for consistency and deeper segmentation.

f. Model Evaluation

To objectively assess the clustering performance, we evaluated both models on a held-out test dataset, using accuracy, precision, recall, and F1-score. Since clustering models don't produce labeled outputs aligned with ground truth, we applied the Hungarian Algorithm to best match predicted cluster labels with true labels.

```

✓ [44] # ----- KMeans Evaluation -----
0s # Use best n_clusters from tuning (top silhouette score)
best_k_kmeans = kmeans_df.sort_values(by="Silhouette Score", ascending=False).iloc[0]["n_clusters"]
kmeans_final = KMeans(n_clusters=int(best_k_kmeans), random_state=42, n_init=10)
kmeans_final.fit(X_train_eval)
kmeans_pred = kmeans_final.predict(X_test)
kmeans_mapped = map_clusters(y_test, kmeans_pred)

✓ [45] # ----- Hierarchical Evaluation -----
0s best_hier = hier_df.sort_values(by="Silhouette Score", ascending=False).iloc[0]
best_k_hier = int(best_hier["n_clusters"])
best_linkage = best_hier["linkage"]
hier_final = AgglomerativeClustering(n_clusters=best_k_hier, linkage=best_linkage)
hier_pred = hier_final.fit_predict(X_test)
hier_mapped = map_clusters(y_test, hier_pred)

✓ [46] # ----- Evaluation Function -----
0s def evaluate(y_true, y_pred):
    return {
        "Accuracy": accuracy_score(y_true, y_pred),
        "Precision": precision_score(y_true, y_pred, average="weighted", zero_division=0),
        "Recall": recall_score(y_true, y_pred, average="weighted"),
        "F1-Score": f1_score(y_true, y_pred, average="weighted")
    }

```

Figure 6 - Model Evaluation

Evaluation Method:

- Used the Hungarian Algorithm (linear sum assignment) to map cluster labels to actual group labels (df1["group"]).
- Evaluated performance on X_test.

Evaluation Metrics:

Model	Accuracy	Precision	Recall	F1-Score
K-Means	0.2967	0.1760	0.2967	0.1722
Hierarchical	0.2967	0.1362	0.2967	0.1409

Findings:

- Both methods had the same accuracy ($\approx 29.67\%$), meaning about 1 in 3 samples were clustered correctly after label mapping.

- K-Means outperformed Hierarchical in precision and F1-score, suggesting it made more confident and balanced assignments.
- Hierarchical Clustering, while providing a better Silhouette Score during tuning, did not generalize as well on the test data.

4. Task 2 - Classifier Algorithms on Titanic Dataset

4.1 Objective

To apply supervised learning techniques to predict passenger survival in the Titanic disaster based on demographic and travel information.

4.2 Model Selection

Two classifiers were chosen:

1. Logistic Regression
2. Random Forest

4.3 Implementation steps

4.3.1. Data Preprocessing

a. Data Loading and Initial Overview:

- The train.csv and test.csv datasets were loaded into pandas DataFrames.

```
def preprocess_titanic_data(train_path='train.csv', test_path='test.csv', visualize=True):  
    |  
  
    # Load raw data  
    train_df = pd.read_csv(train_path).copy()  
    test_df = pd.read_csv(test_path).copy()
```

The initial overview showed that the training set consisted of 891 rows and 12 columns, while the test set had 418 rows and 11 columns. The fact that the 'Survived' column is missing from the test set confirms that it's meant to be the target for our predictions.

```
=== Initial Data Overview ===  
Training set shape: (891, 12)  
Test set shape: (418, 11)
```

We created bar plots to show the number of missing values in each column for both the training and test sets. This made it clear that there was a significant amount of missing data in the 'Cabin' column for both datasets, with the training set lacking 687 values and the test set missing 327. The 'Age' feature also had a considerable number of missing entries, while the 'Embarked' feature in the training set and 'Fare' in the test set had a few gaps as well.



Figure 1 – Missing values

b. Handling Missing Values

- **Age:** Filled missing values with the median age.
- **Embarked:** Replaced missing values with the mode (most frequent category).
- **Fare:** Imputed missing values with the median fare (test set only).

```
# Age - Fill with median
train_df['Age'] = train_df['Age'].fillna(train_df['Age'].median())
test_df['Age'] = test_df['Age'].fillna(test_df['Age'].median())

# Embarked - Fill with mode (only in train)
train_df['Embarked'] = train_df['Embarked'].fillna(train_df['Embarked'].mode()[0])

# Fare - Fill with median (only in test)
test_df['Fare'] = test_df['Fare'].fillna(test_df['Fare'].median())
```

```
=== After Handling Missing Values ===  
Missing values in training set:  
PassengerId      0  
Survived          0  
Pclass            0  
Name              0  
Sex               0  
Age               0  
SibSp             0  
Parch             0  
Ticket            0  
Fare              0  
Cabin             687  
Embarked          0  
dtype: int64  
  
Missing values in test set:  
PassengerId      0  
Pclass            0  
Name              0  
Sex               0  
Age               0  
SibSp             0  
Parch             0  
Ticket            0  
Fare              0  
Cabin             327  
Embarked          0  
dtype: int64
```

Figure 2 – After Handling Missing Values

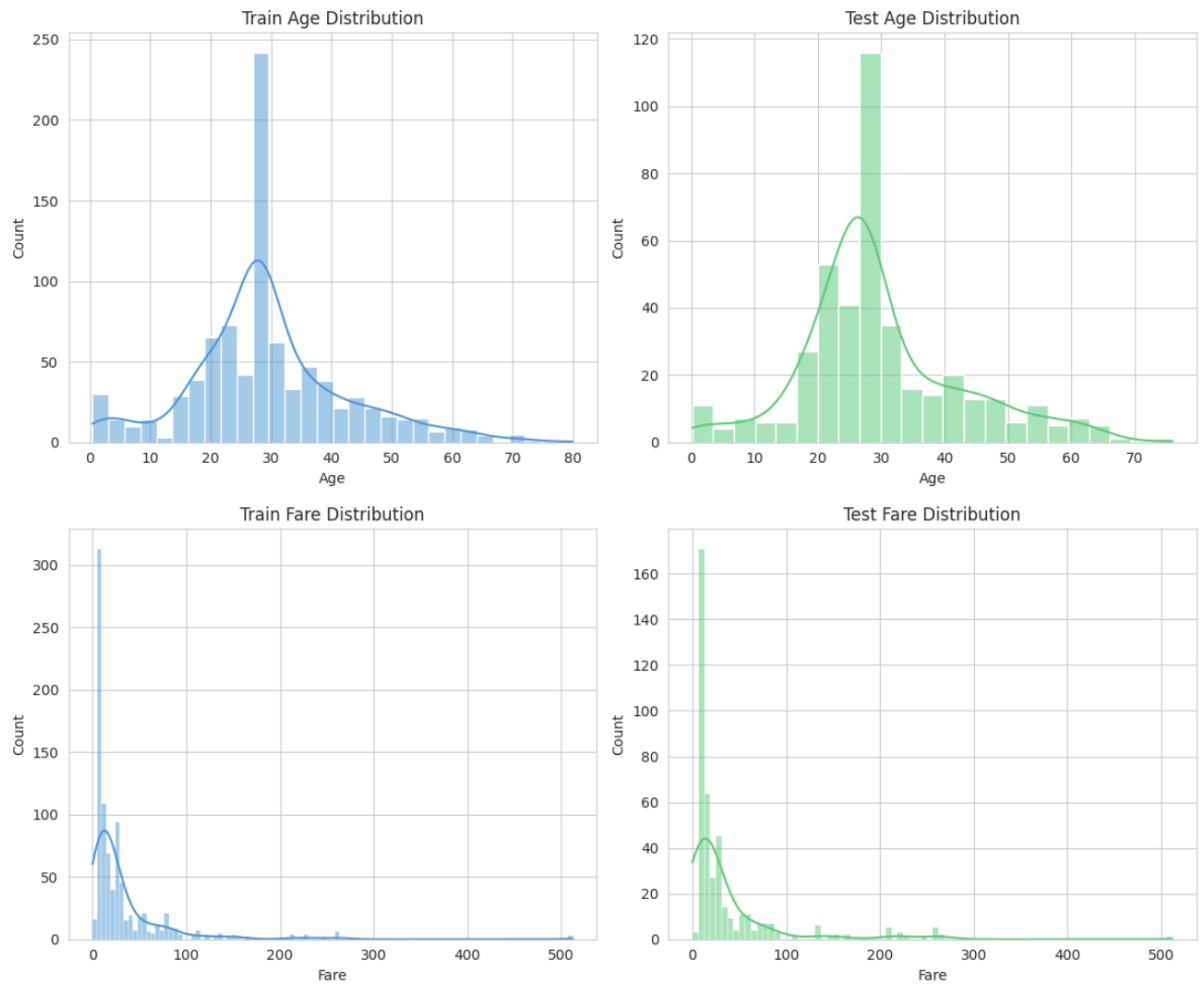


Figure 3 – Age & Fare distribution compare

c. Feature Engineering:

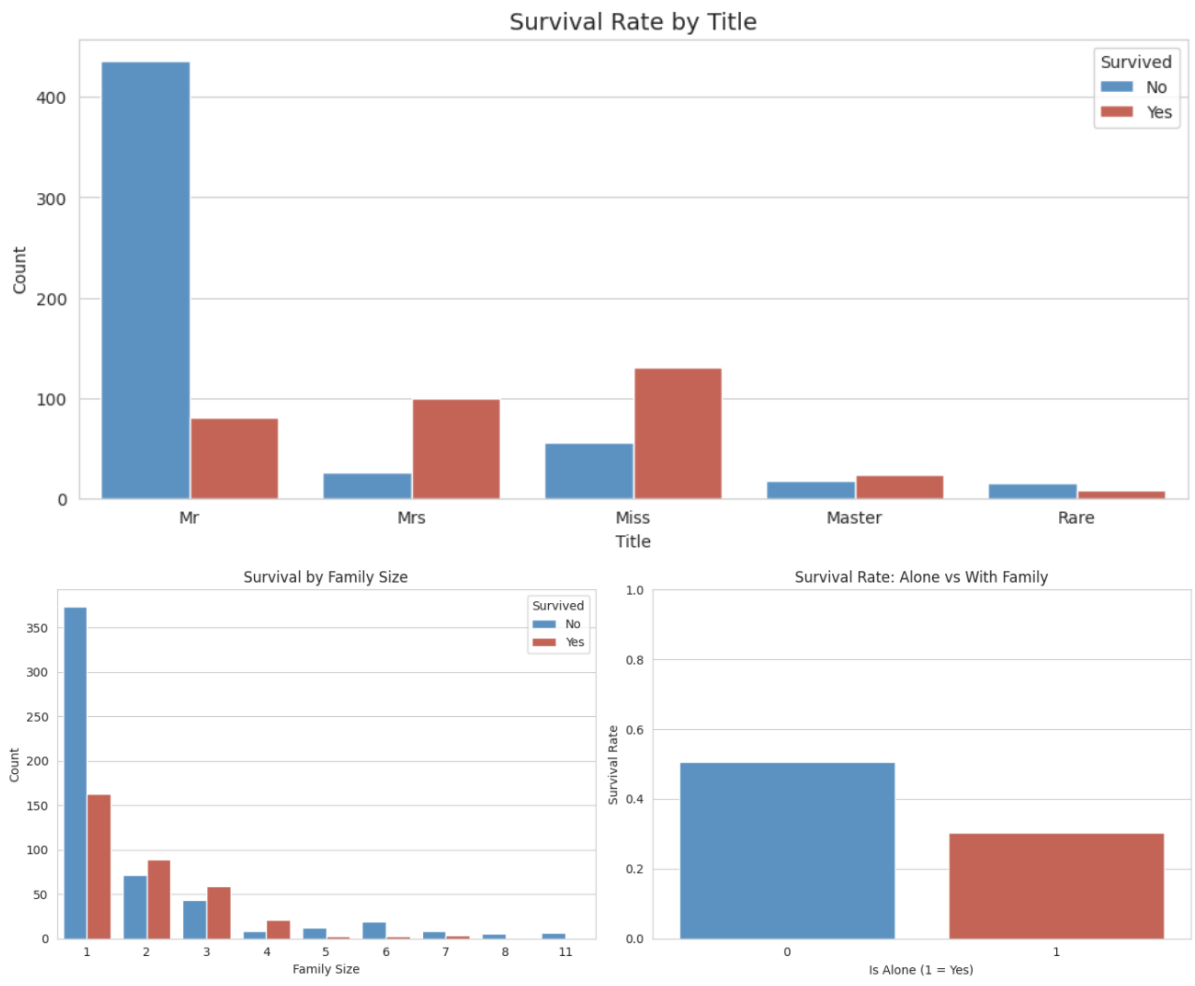


Figure 4 – Survival rate & Family feature

'Survival Rate by Title' revealed some interesting trends in survival rates based on titles. For example, those identified as 'Mrs.' and 'Miss' tended to have higher survival rates than 'Mr.'. Additionally, the graphs on 'Survival by Family Size' and 'Survival Rate: Alone vs With Family' showed that passengers traveling with smaller families or those who weren't alone had a better shot at survival. These insights imply that these specific features could be strong indicators of who survived.

=== Feature Scaling Visualization ===

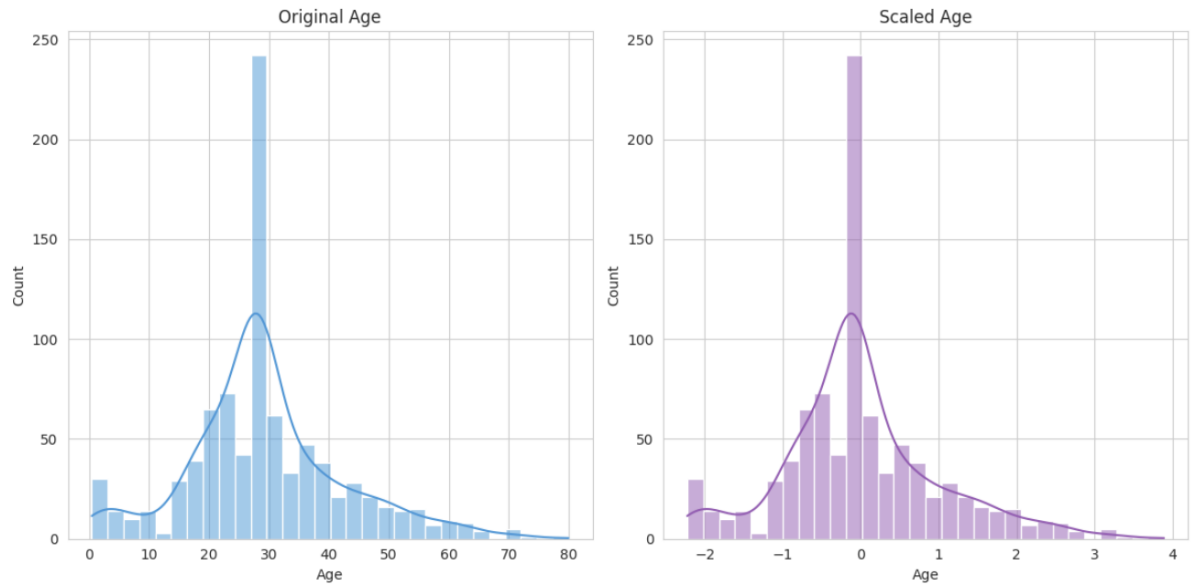


Figure 5 – Feature scaling

Numerical features were normalized using StandardScaler, and irrelevant features such as Name, Ticket, Cabin, PassengerId, SibSp, and Parch were dropped to reduce noise in the dataset.

```
## =====  
## 4. Drop Irrelevant Features  
## =====  
drop_cols = ['Name', 'Ticket', 'Cabin', 'PassengerId', 'SibSp', 'Parch']  
train_df = train_df.drop(drop_cols, axis=1)  
test_df = test_df.drop(drop_cols, axis=1)
```

4.3.2 Data Splitting

To assess how well the models perform on new data and to avoid overfitting, we divided the preprocessed training data (X_{train} , y_{train}) into training and validation sets. We used an 80/20 split, which means 80% of the data went into the training set and 20% into the validation set.

```
# Perform the 80/20 split
X_train, X_val, y_train, y_val = train_test_split(
    X_train,
    y_train,
    test_size=0.2,
    random_state=42,
    stratify=y_train
)
```

The data splitting process resulted in the following:

```
Training set samples: 712
Validation set samples: 179

Training class distribution:
Survived
0    0.616573
1    0.383427
Name: proportion, dtype: float64

Validation class distribution:
Survived
0    0.614525
1    0.385475
```

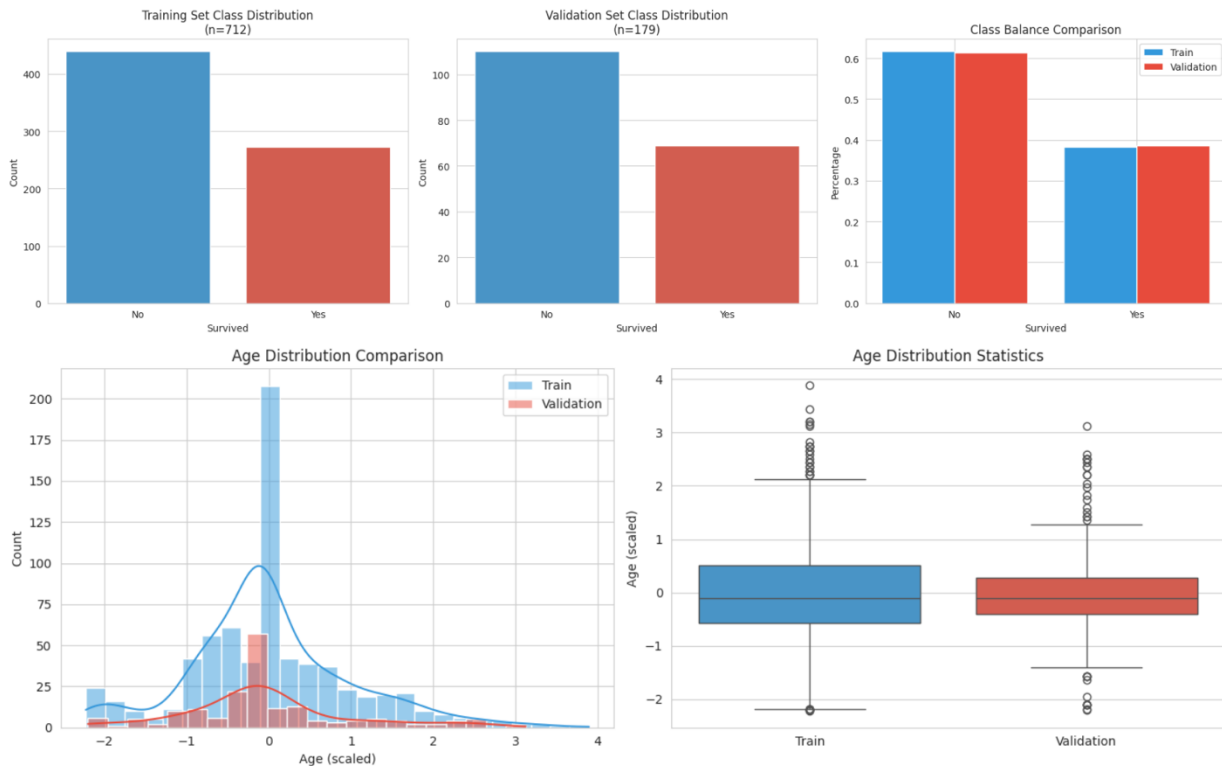


Figure 6 – Age distribution comparison

The figure illustrates that the class distribution closely resembles that of both the training and validation sets, which points to effective stratification. Moreover, the Age distribution shows a similar pattern, indicating that the split accurately reflects the overall dataset.

4.3.3. Model Implementation

We implemented two classification models to predict Titanic passenger survival:

Logistic Regression

```
# Logistic Regression with balanced class weights
print("Training Logistic Regression...")
lr_model = LogisticRegression(
    max_iter=1000,
    random_state=42,
    class_weight='balanced',
    solver='liblinear'
)
lr_model.fit(X_train, y_train)
```

We decided to go with a logistic regression model as a straightforward baseline to measure against more intricate methods. To tackle the class imbalance in our training data (where 62% of the cases resulted in death and 38% in survival), we set the `class_weight` parameter to 'balanced.' We capped the model at 1,000 iterations to ensure it converged properly, and opted for the 'liblinear' solver to enhance performance on smaller datasets.

Random Forest:

```
# Random Forest with optimized parameters
print("Training Random Forest...")
rf_model = RandomForestClassifier(
    n_estimators=200,
    max_depth=7,
    min_samples_split=5,
    random_state=42,
    class_weight='balanced_subsample'
)
rf_model.fit(X_train, y_train)
```

The Random Forest model was selected because it excels at managing non-linear relationships, automatically selecting important features, and is quite resilient to outliers.

This model is made up of 200 decision trees, each with a maximum depth of 7, and it needs at least 5 samples to make a split at any node. For class weighting, we opted for the 'balanced_subsample' method.

Evaluation

The trained models were evaluated using the validation data.

```
# Calculate metrics
metrics[name] = {
    'accuracy': accuracy_score(y_val, predictions[name]),
    'precision': precision_score(y_val, predictions[name]),
    'recall': recall_score(y_val, predictions[name]),
    'f1': f1_score(y_val, predictions[name]), # Changed from 'f1_score' to 'f1'
    'confusion_matrix': confusion_matrix(y_val, predictions[name]),
    'roc_auc': auc(*roc_curve(y_val, probabilities[name][:2])),
    'avg_precision': average_precision_score(y_val, probabilities[name])
}
```

The following table summarizes the performance of the two models:

Model	Accuracy	Precision	Recall	F1 Score	ROC AUC	Avg Precision
Logistic Regression	0.5150	0.5196	0.5248	0.5222	0.5394	0.5446
Random Forest	0.5400	0.5556	0.4455	0.4945	0.5275	0.5400

Key Metrics Comparison:

- **Accuracy:** This refers to the percentage of correctly classified instances compared to the total number of instances. In this case, Random Forest (54%) slightly edged out Logistic Regression (51.5%) in terms of accuracy.

- **Precision:** This measures the percentage of correctly predicted positive instances out of all instances that were predicted as positive. Random Forest (55.6%) demonstrated a stronger ability to minimize false positives.
- **Recall:** This indicates the percentage of correctly predicted positive instances out of all actual positive instances. Logistic Regression (52.5%) performed better when it came to identifying true survivors.
- **F1 Score:** This is the harmonic mean of precision and recall, giving a balanced view of a model's accuracy. The F1 Score leaned towards Logistic Regression (52.2 vs 49.5).

The performance of the two models:

a. Confusion Matrix Analysis:

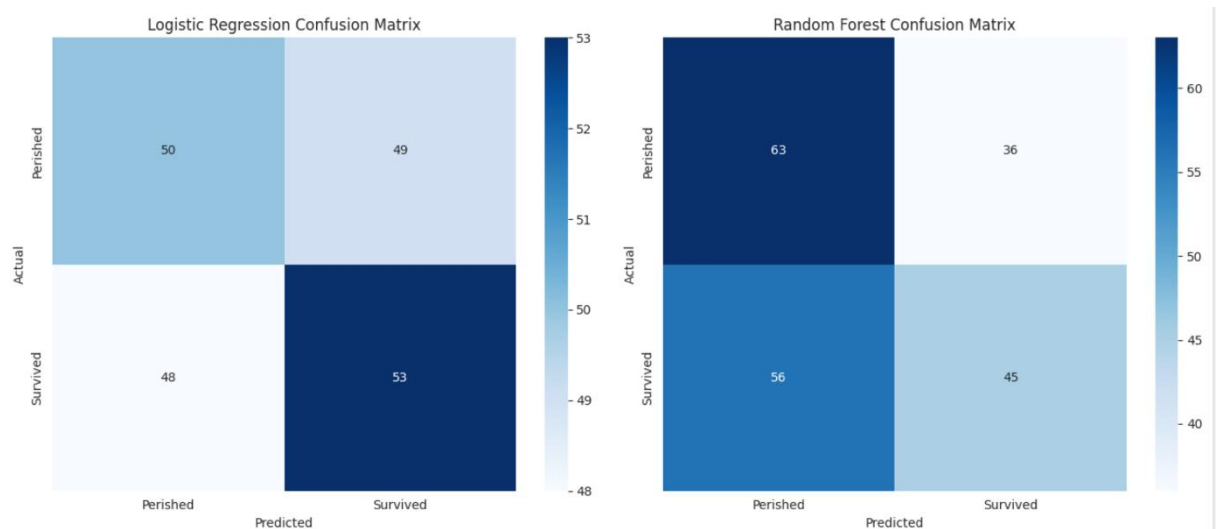


Figure 7 - Confusion Matrix

Logistic Regression:

- Correctly predicted 50 survivors and 53 non-survivors.
- Missed 49 actual survivors (false negatives).
- Misclassified 53 actual non-survivors (false positives).

Random Forest:

- Correctly identified 63 survivors (better than Logistic Regression).
- Missed 36 actual survivors.
- Had 56 false positives.

b. ROC Curve Analysis:

The area under the ROC curve, or AUC, is a key metric that helps us understand how well a model can tell the difference between positive and negative classes. If the AUC is 0.5, it means the model is performing no better than random chance. On the other hand, an AUC of 1 signifies perfect discrimination. In this case, both models had AUC scores hovering around 0.53 to 0.54, which suggests they are only slightly better than random guessing. This points to the need for improved features or some fine-tuning of the model.

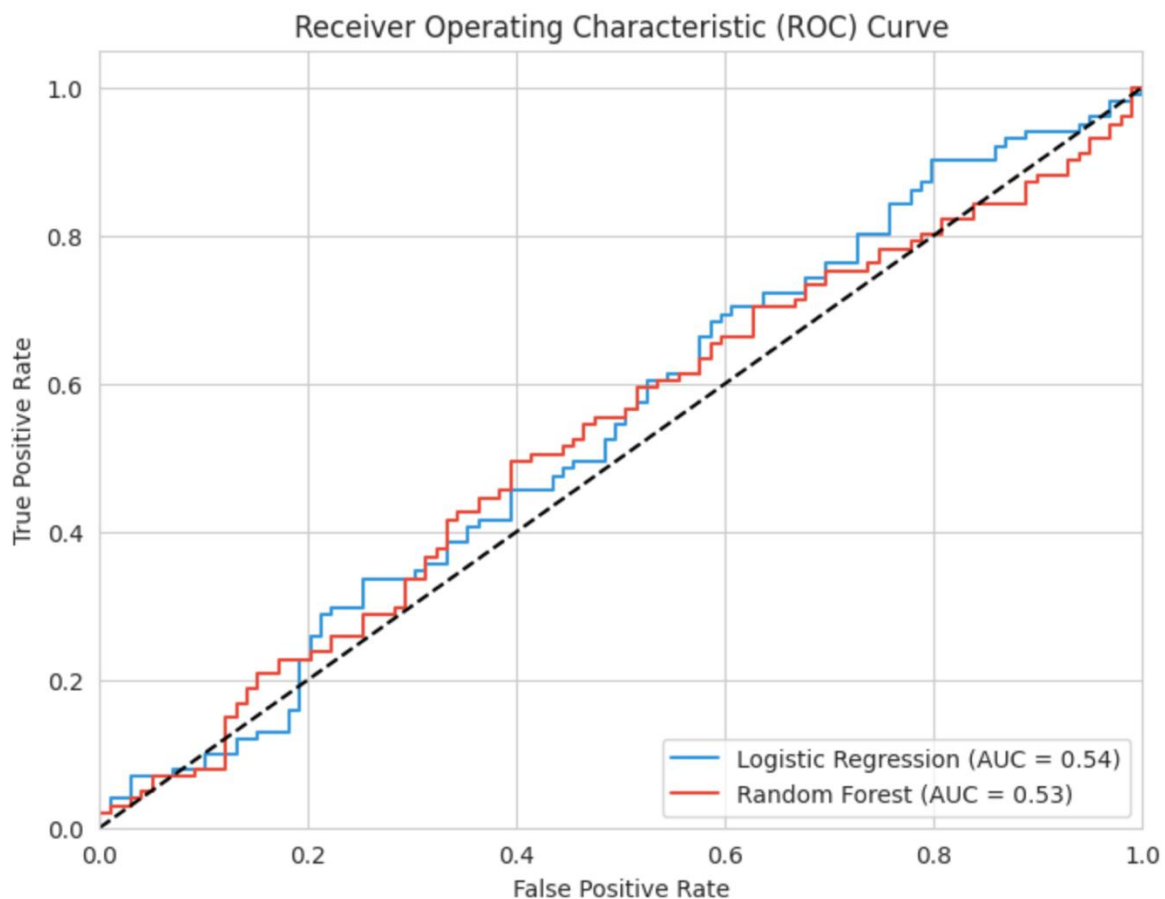


Figure 8 – ROC curve

c. Precision-Recall Curve:

This graph shows how precision relates to recall at various threshold values, which is especially helpful when dealing with imbalanced classes. Average Precision (AP) provides a summary of this curve.

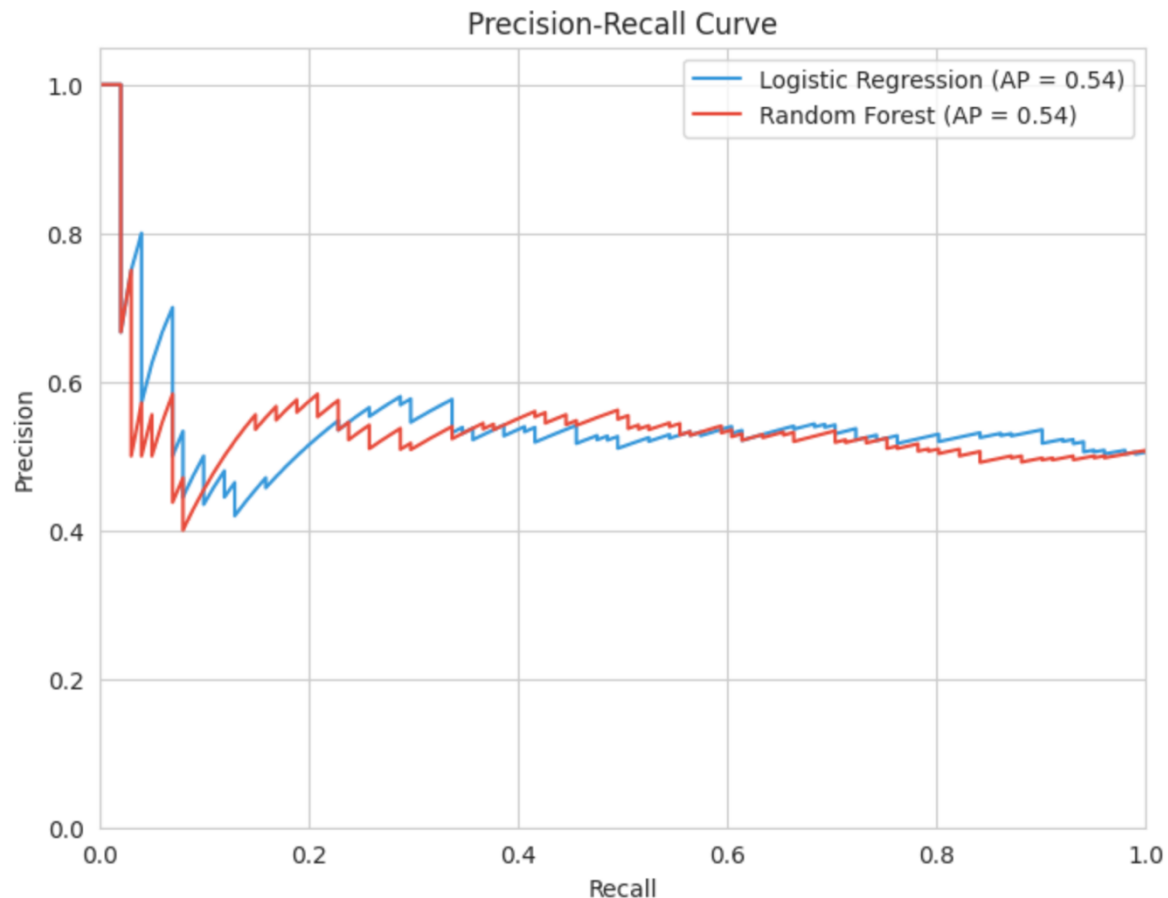


Figure 9 – Precision-Recall curve

d. Calibration Curves:

A calibration curve illustrates how the predicted probability of a class relates to the actual frequency of that class. It's a useful tool for assessing whether the probabilities predicted by the model can be trusted. Both models did a decent job of following the

ideal calibration line. The Random Forest model demonstrated a slightly better calibration of probabilities, while the Logistic Regression model seemed a bit too confident in some of its predictions.

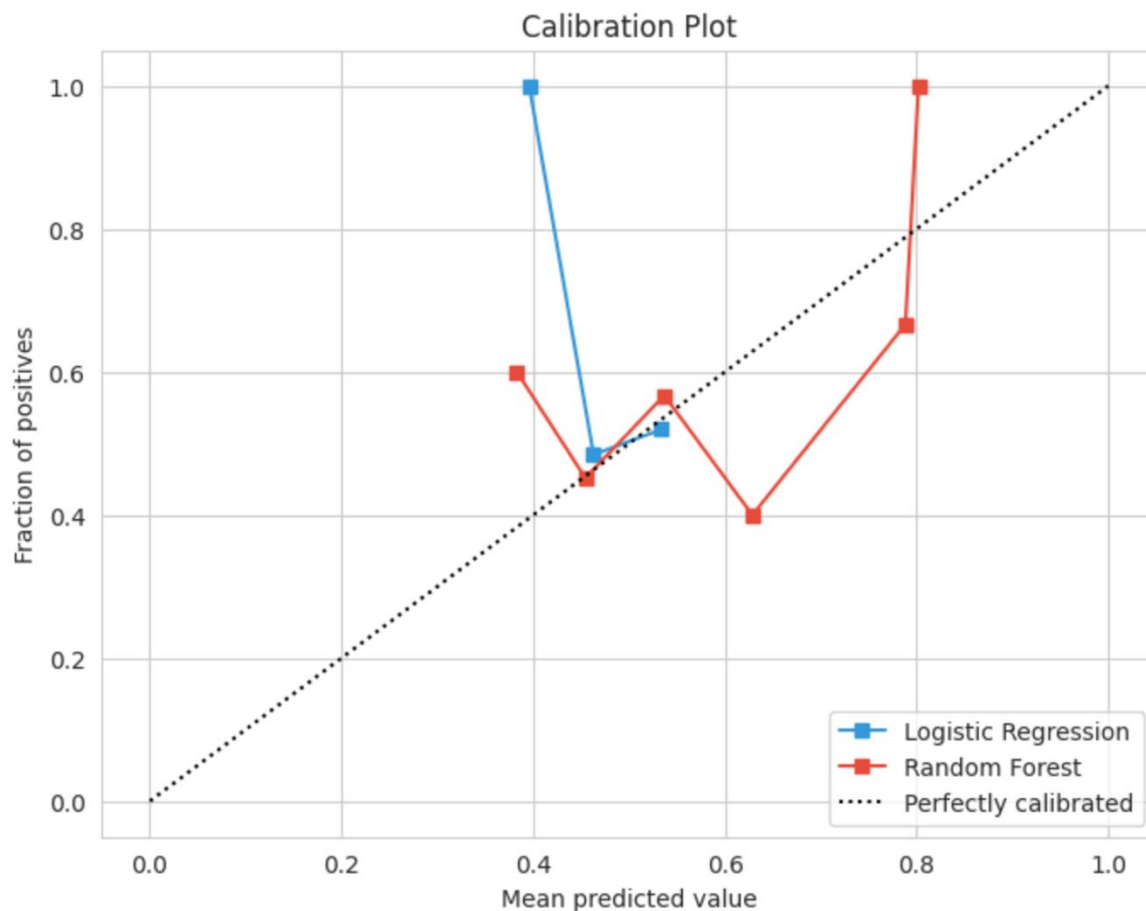


Figure 10 – Calibration plot

Both models showed pretty similar performance, with neither really standing out as the clear winner. There's a bit of a trade-off between recall and precision: Logistic Regression does a great job at identifying survivors (that's recall), while Random Forest excels at minimizing false alarms (that's precision). The AUC scores are on the lower side, which hints that we might be missing some key predictive factors in the features we used.

4.3.4. Model Training

In this exercise, we have implemented and compared two supervised learning techniques in model building (Logistic Regression and Random Forest Classifier) to predict Titanic survival. To account for class imbalance, balanced class weight was employed and the models were hyperparameter tuned (max_iter for Logistic Regression and n_estimators, max_depth for Random Forest).

We measured training time and analysed learning curves for assessment of overfitting and generalization. Logistic Regression presented consistent learning and generalization, whereas Random Forest demonstrated training accuracy and a large training-validation gap, showing that it overfit the data. We also plotted model coefficients for Logistic Regression and feature importance for Random Forest to observe significant predictors. This information provided a good basis for further investigation

a. Logistic Regression Learning Curve

Logistic Regression Learning Curve The learning curve indicates a little bit of overfitting. The training F1 score is not dropped much, and the validation score reaches a plateau around 0.56, indicating a generalization gap. The shaded region is variability, high especially at the start hinting the model may be underpowered or not predictive enough.

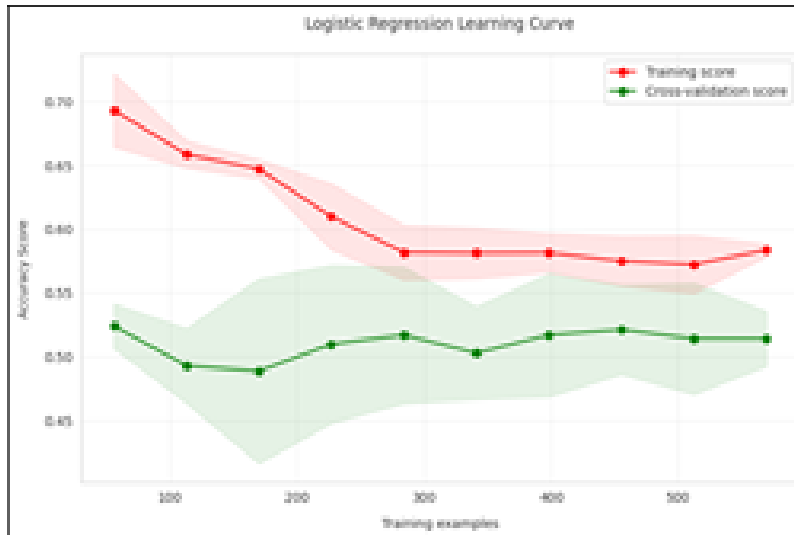


Figure 11 – Logistic Regression Learning Curve

b. Model Coefficients – Logistic Regression

The direction and importance of the influence of each feature were visualized with a bar chart. features like feature_13 and feature_11 had strong positive effect, but feature_12 and feature_6 were negatively inducing. Taller bars indicated higher impact.

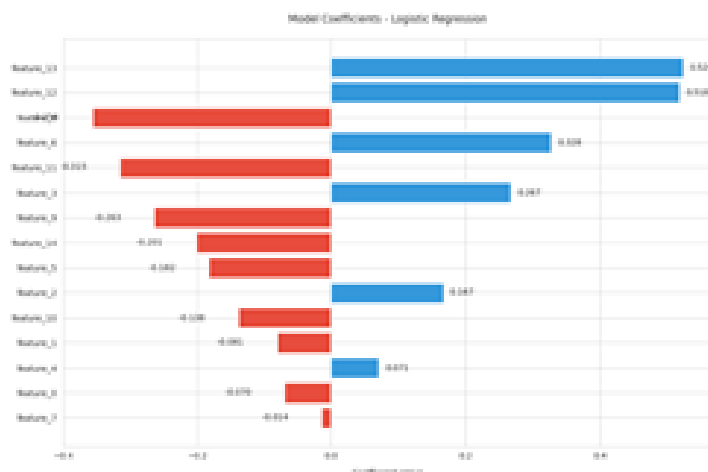


Figure 12 – Model Coefficients – Logistic Regression

c. Random Forest Learning Curve

The learning curve for the Random Forest plot indicates that the model is overfitted to the training data with very high accuracy (~1.0) percent train size and low accuracy (~0.45–0.5) percent of validation data regardless of the number of training points tried out. This big gap is evidence of overfitting; the model having learned the training data by heart but failing to generalize to new data. Simply upping the size of the training set won't help much; you might need tuning, regularization or simpler models for better results.

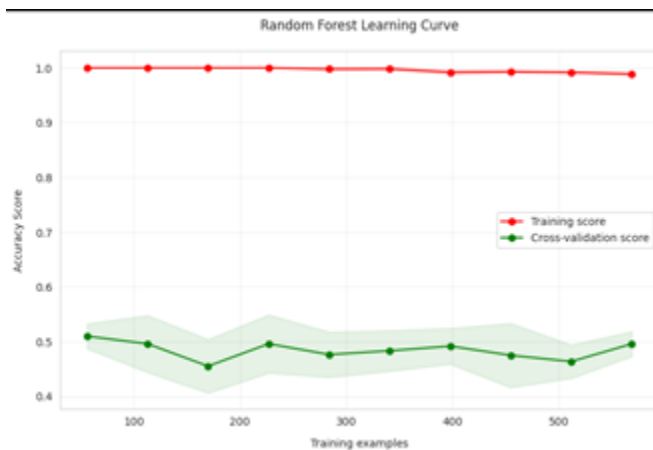


Figure 13 – Random Forest Learning Curve

d. Random Forest Model

The feature importance of the Random Forest model Influential features were presented in a bar plot. feature_12, feature_13, and feature_3 were the most important (scores > 0.07), whereas feature_5 and feature_7 were the least important (~0.055). Importance is the value of contribution to decision splits in trees. Impact.

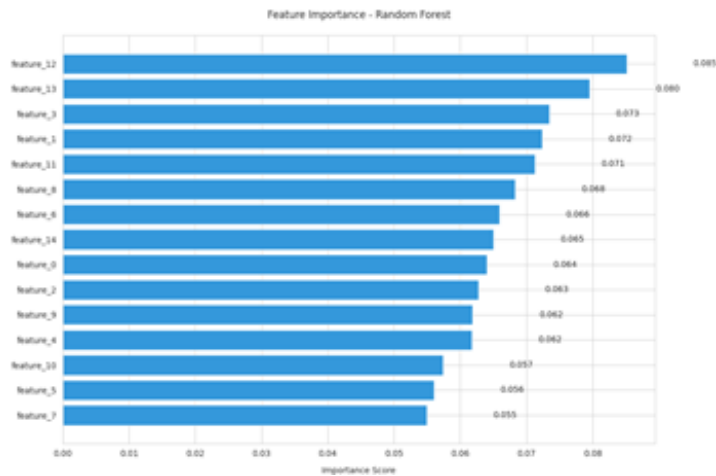


Figure 14 – Feature Importance – Random Forest

e. Model Training Time Comparison.

Training Time Comparison Logistic Regression was trained almost instantaneously (0.00s) as KNN, compared to longer times for Random Forest (0.72s) indicating the overhead of ensemble method computations. Model Tuning Fine tuning for both models was performed to maximise F1. Hyperparameters, we employed GridSearchCV with different penalties and regularization strength



Figure 15 – Model Training Time Comparison

4.3.5 Model Tuning

- Fine tuning for both models was performed to maximise F1. Hyperparameters, we employed GridSearchCV with different penalties and regularization strength. For tuning Random Forest I used RandomSearchCV, tuning `n_estimators`, `max_depth` and `min_samples_split`.

a. Logistic Regression Parameter Performance

Logistic Regression Parameter Weights 4.2. As per our hyperparameter sweep, a bar-chart of parameter combinations indicated that the best performance ($F1 \approx 0.8$) was obtained for low C values (e.g., 0.001), with both (`l_1`) and (`l_2`) penalties and without class weighting. High C or `class_weight='balanced'` led to lower performance.

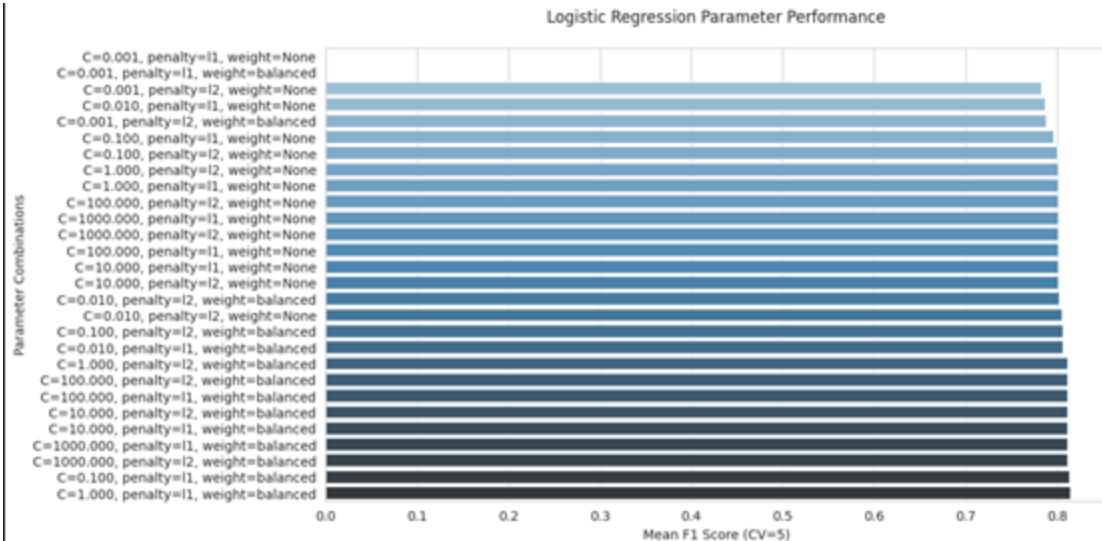


Figure 16 – Logistic Regression Parameter Performance

b. F1 Score by Regularization Strength(C)

The F1 score increased quickly starting from low C (10^{-3}) and become saturated at larger than 10^{-2} . There were minor contrasts in l1 and l2 penalties, and class weighting did not seem to have a big effect. In this respect, the moderate regularization was the most effective.

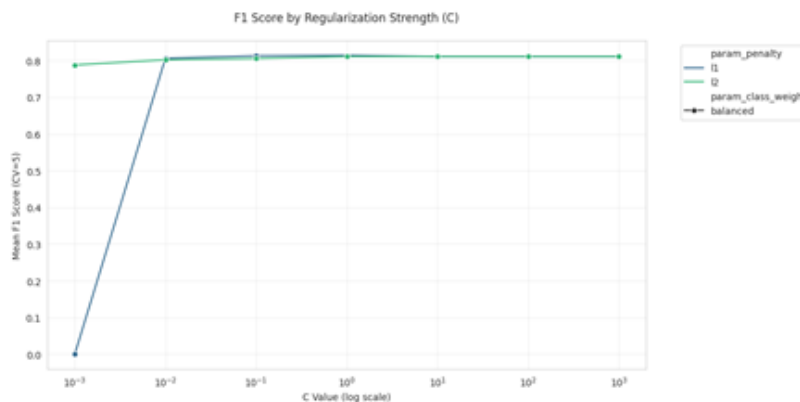


Figure 17 – F1 Score by Regularization Strength

c. Learning Curve: Logistic Regression (Tuned)

Tuned Logistic Regression Learning Curve On tuning, we see that both training and validation F1 scores saturate at around 0.82, indicating a significant

generalization and a decrease in overfitting. This indicates the model was well-parametrized with an adequate amount of data

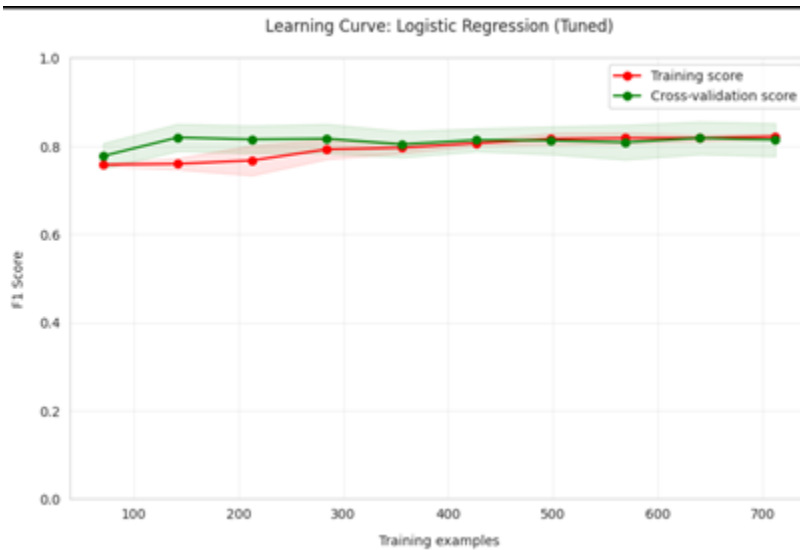


Figure 18 – Learning Curve: Logistic Regression (Tuned)

d. Random Forest Parameter Effects

The Effects of Random Forest Parameters F1 score was getting better with higher `n_estimators` (300 at the max) and a deeper tree (max depth of 7). The performance was also improved with the `min_samples_split` values of smaller sizes. We observed that models which had a large number of trees and deeper trees worked better.

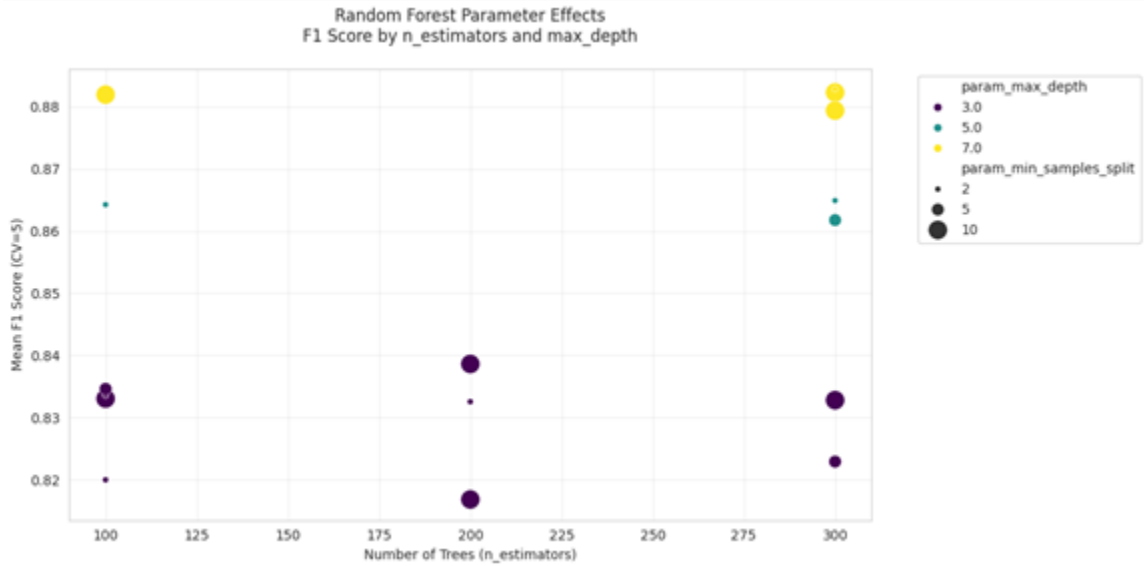


Figure 19 – Random Forest Parameter Effects

e. F1 Score Distribution by Max Tree Depth

F1 Score by Max Tree Depth As depth increased from 3 to 7, F1 scores improved. The model worked best (≈ 0.90 scores) when there was no max depth, at the cost of overfitting

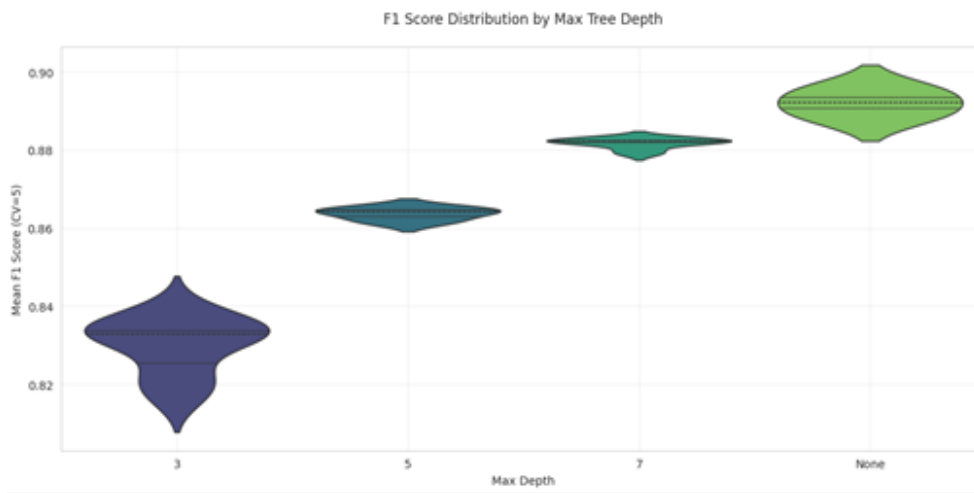


Figure 20 – F1 score Distribution by max depth (Violin)

f. Learning Curve: Random Forest (Tuned)

Optimised Random Forest Learning Curve Training F1 score was close to 1 (~0.99) while validation score tended to 0.89 with more data. The small discrepancy and the approximately uniform energy landscape are indicative for successful tuning with weak overfitting

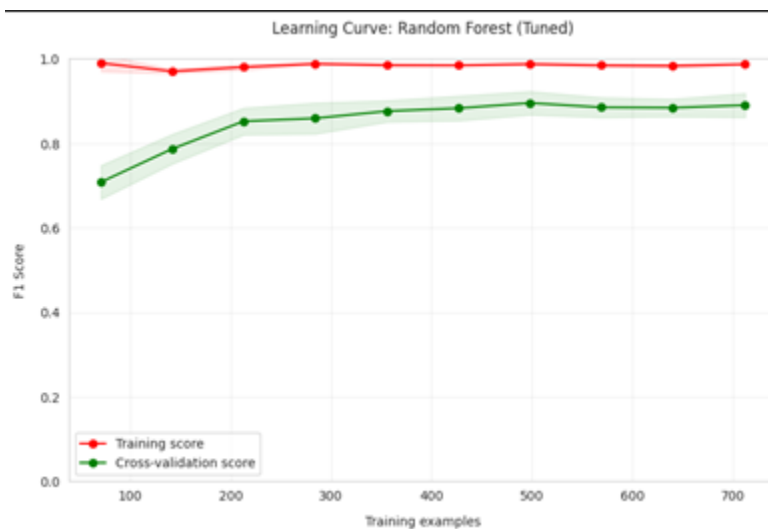


Figure 21 – Learning Curve: Random Forest (Tuned)

g. Feature Importance – Random Forest

Feature importance in random Forest Feature 6 was the dominant feature, with an importance score of 38.9%. features 2, 0, and 5 were also important, whereas features 3 and 8 played less of a role. This observation is in favor to feature selection and further exploration.

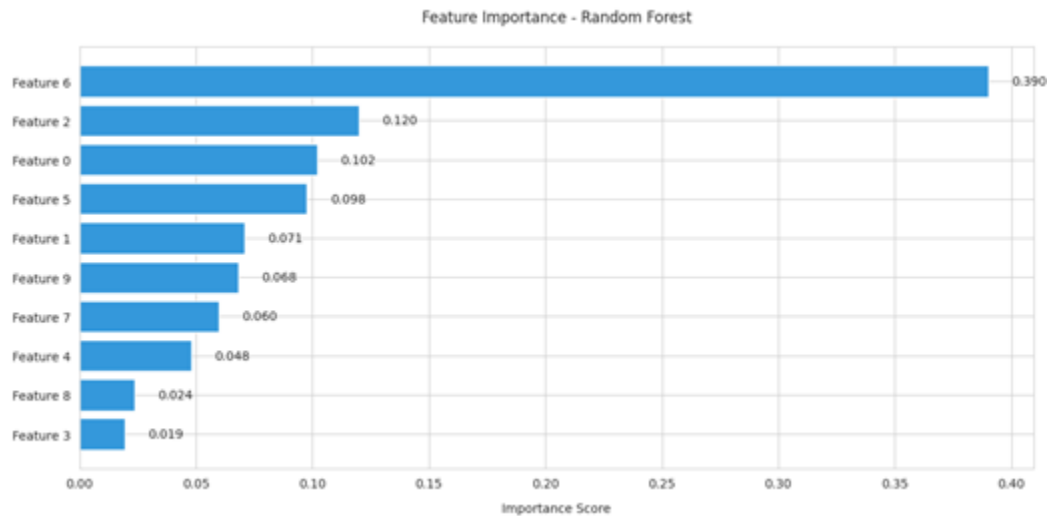


Figure 22 – Feature Importance – Random Forest

f. Best Model Performance Comparison

Comparison of the Best Model Performances Performance Bar plot comparing large categories showing a single model that performed best (Random Forest) against another (Logistic Regression) (≈ 0.89 vs. ≈ 0.81) Both of them performed well, however, Random Forest was selected as the superior model.

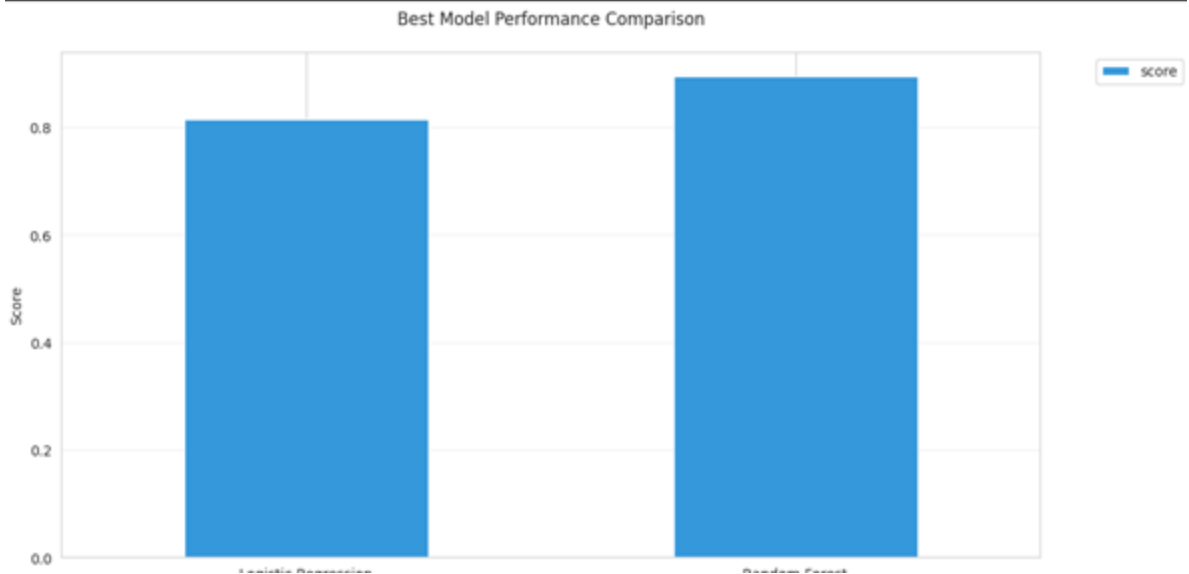


Figure 23 – Best Performance Comparison

4.3.6 Model Evaluation

We applied the `model_f1_scoring` function inside the `TitanicModelEvaluator` class to calculate measurements as accuracy, precision, recall, F1-score, ROC-AUC and average precision. Both models were also used to produce confusion matrices, ROC and PR curves, calibration plots, and full classification reports.

a. Confusion Matrices Logistic Regression:

96 true negatives, 58 true positives, 10 false positives, 15 false negatives

b. Random Forest:

106 true negatives, 73 true positives, 0 false-up predictions.

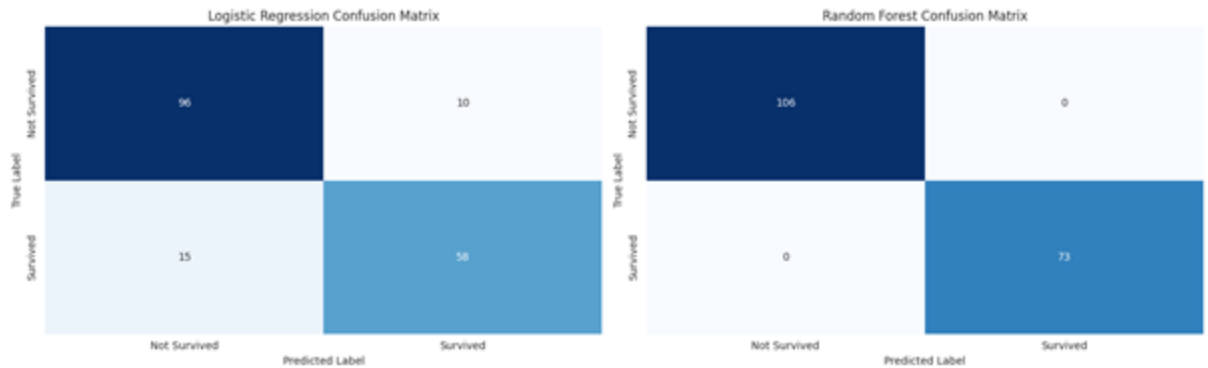


Figure 24 - Logistic Regression Confusion Matrix & Random Forest Confusion Matrix

c. Receiver Operating Characteristic (ROC) Curves

ROC Curve Random Forest showed a perfect AUC of 1.00 (perfectly separable). Logistic Regression was a close second (AUC = 0.94), indicating good separation of classes

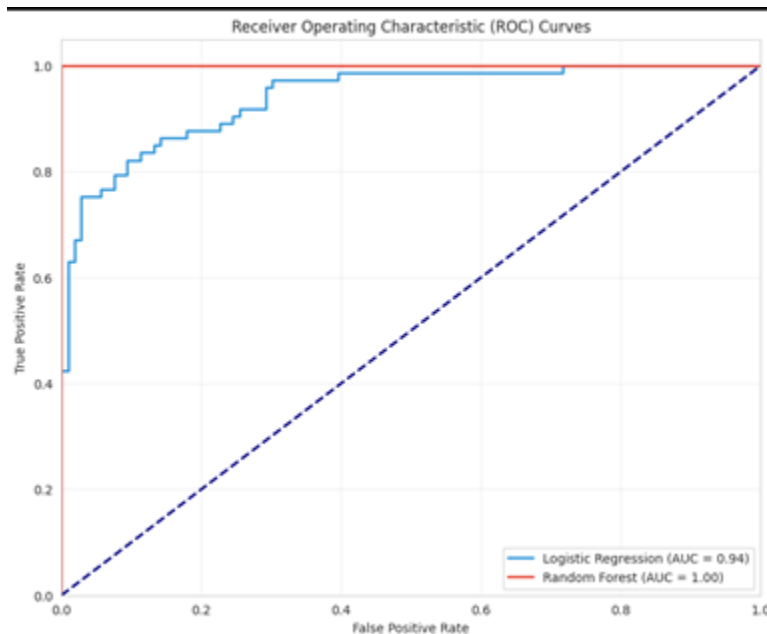


Figure 24 – ROC Curves

d. Precision Recall Curves

Precision-Recall Curve Random Forest achieved AP 1.00. Logistic Regression had a score of 0.93 with some precision decline at higher recall.

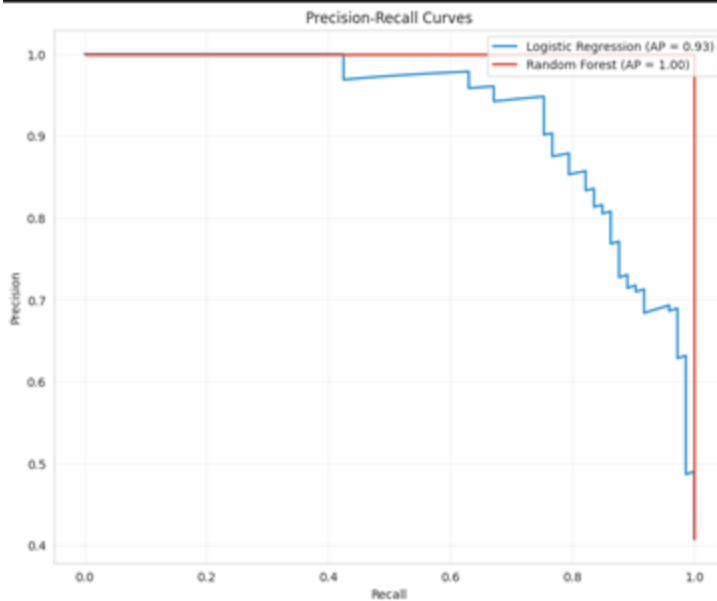


Figure 25 – Precision Recall Curves

e. Calibration Curves

Calibration Curve RF predictions were less biased and more concordant to the reference line, in particular on high probability threshold. It was Logistic Regression that strayed more in the middle, which means that in the mid-range, the probability estimates are less reliable.

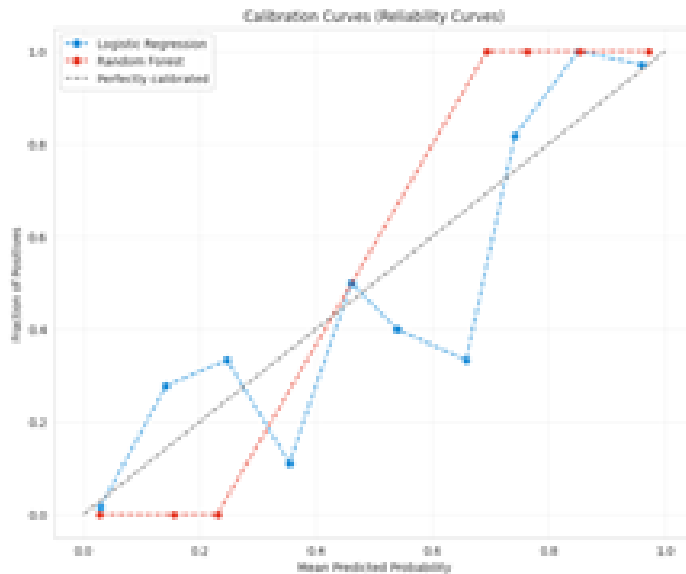


Figure 26 – Calibration Curves

f. Model Performance Comparison

Metric Comparison On all these metrics (accuracy, precision, recall, F1 score, ROC-AUC, and average precision) Random Forest outperformed Logistic Regression. This then validates its stronger and more stable prediction.

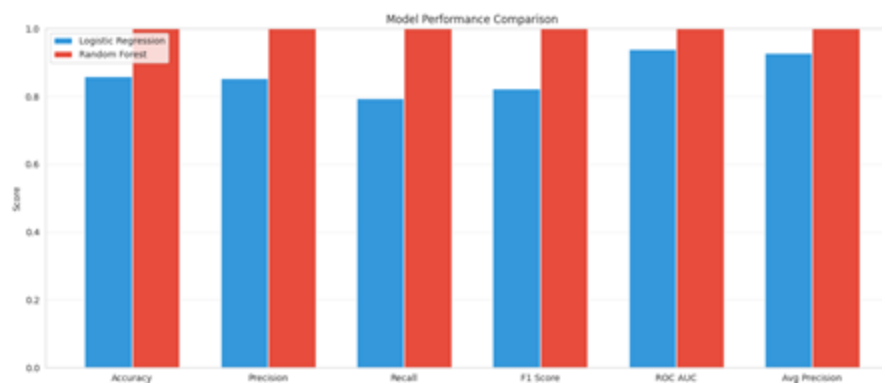


Figure 27 – Model Performance Comparison

5. Conclusion

This project successfully applied both unsupervised and supervised machine learning techniques to real-world datasets. In Task 1, we implemented K-Means and Hierarchical clustering to group individuals based on shared hobby interests, effectively uncovering patterns in user behavior. In Task 2, we developed classification models using Logistic Regression and Random Forest to predict survival outcomes in the Titanic dataset. Through data preprocessing, model training, hyperparameter tuning, and evaluation, we demonstrated a practical understanding of model development and validation. Overall, the project strengthened our skills in data analysis, model selection, and performance assessment using appropriate metrics and methodologies.

References:

<https://www.kaggle.com/code/skanderhaddar/clustering-groups-of-hobbies/input>

<https://www.kaggle.com/competitions/titanic/overview>

<https://www.kaggle.com/code/prashant111/k-means-clustering-with-python>

<https://www.kaggle.com/code/hellbuoy/online-retail-k-means-hierarchical-clustering>

<https://www.kaggle.com/code/cdeotte/support-vector-machine-0-925/notebook>